

Luan Le

Full Stack Engineer

+358 41 751 6313 | thanhluan081999@gmail.com | linkedin.com/in/luan-le-93a89b218 | github.com/ELNAUL99

Application: Member of Technical Staff

Dear Founders,

For three years my software had a physical failure mode. If the booking system went down, guests stood in the doorway of a restaurant in Helsinki while staff hunted for a paper list. If the point-of-sale broke mid-service, someone was standing at the till with a card in their hand waiting for me. I was the only technical person there, so there was nobody to escalate to and no version of the problem that stayed abstract. That experience is the reason your line about caring what the software actually does in the world was the part of this posting I kept coming back to. City logistics has the same property at a far larger scale: when the system is wrong, something physical in the world goes to the wrong place.

The clearest thing I built there was a booking system that now handles over 200 reservations a month and cut customer booking time by more than half. What makes it worth mentioning is not the feature but what it replaced: a manual phone and email process that staff had been absorbing by hand, with all the coordination happening in someone's head. Finding the real problem took longer than building the solution. I had assumed people abandoned the old process because it was slow, but watching what they actually did showed they were giving up because they could not tell whether their booking had gone through. The problem was confidence, not speed. Since then I treat the first version of any requirement as a symptom rather than a specification, which I suspect is most of what product sense means in practice.

On the AI side, my main work is ReguLens, a seven-stage agent pipeline that produces cited compliance opinions on the EU AI Act, which won the Norrin Challenge at the Aalto AI hackathon. The architecture is the part relevant to you rather than the domain. Each stage produces a distinct persistent artefact, so when output is wrong I can trace it to the stage that caused it instead of debugging one opaque blob. Typed schemas act as the contract at every boundary, so malformed output fails where it is produced rather than propagating. A deterministic checker rejects any citation absent from the source material, and a critique agent argues the opposing conclusion before anything is accepted. A cheap model gates whether the expensive one runs at all. Every external call has timeout, retry, and provider fallback, because I assumed from the start that an upstream dependency would eventually degrade rather than fail cleanly. Those are the same instincts anomaly detection and agent tooling need in an operational system.

Beyond that I work across the stack in TypeScript, React, Next.js, Node, Python, and PostgreSQL, build and consume REST APIs, run Docker, and gate every merge on typechecking, linting, automated tests, and static security analysis. I have built integrations against third-party APIs and document ingestion pipelines, though I should say plainly that I have not worked with ERP systems or carrier APIs before. What I do have is three years of being the person who makes an integration work when the documentation is wrong and the vendor is slow to reply.

What appeals to me here is the size of the team and the closeness to the operation. I do my best work when I own something end to end, when I can talk to the people actually using it, and when the consequences are visible. Working directly with founders on architecture from day one, in a domain where the software moves real things through real cities, is the version of this job I would most want. I build small games in my own time for a similar reason: I like making things and finding out where they break.

I would be glad to talk about how I could contribute.

Kind regards,

Luan Le